Pandas — LEVEL 9: Missing Values

Complete guide with line-by-line code, outputs, examples, and easy interview explanations.

1. What is Missing Data?

Code

```
import pandas as pd
df = pd.DataFrame({"Name":["Amit","Riya","John"], "Age":[22,None,30]})
print(df)
```

Output

```
   Name   Age
0  Amit  22.0
1  Riya   NaN
2  John  30.0
```

Explanation: Missing data means a value is unavailable or was not recorded.

Easy interview explanation: Missing data is common in real-world datasets and must be handled before analysis.

2. NaN

Code

```
df = pd.DataFrame({"Age":[22,float("nan"),30]})
print(df)
print(df["Age"].isna())
```

Output

```
    Age
0  22.0
1   NaN
2  30.0
0    False
1     True
2    False
Name: Age, dtype: bool
```

Explanation: NaN means Not a Number and is commonly used for missing numeric values.

Easy interview explanation: NaN is a standard marker for missing numeric data.

3. None

Code

```
df = pd.DataFrame({"Name":["Amit",None,"John"]})
print(df)
print(df["Name"].isna())
```

Output

```
   Name
0  Amit
1  None
2  John
0    False
1     True
```

```
2    False
Name: Name, dtype: bool
```

Explanation: None is Python's null value and pandas recognizes it as missing.

Easy interview explanation: None can represent missing values, especially in object/string data.

4. pd.NA

Code

```
df = pd.DataFrame({"Name":pd.Series(["Amit",pd.NA,"John"],dtype="string")})
print(df)
print(df["Name"].isna())
```

Output

```
   Name
0  Amit
1  <NA>
2  John
0    False
1     True
2    False
Name: Name, dtype: boolean
```

Explanation: pd.NA is pandas' dedicated missing-value marker and works with nullable dtypes.

Easy interview explanation: pd.NA is useful for consistent missing-value handling in modern pandas.

5. Detect Missing Values

Code

```
df = pd.DataFrame({
    "Name":["Amit",None,"John"],
    "Age":[22,25,None],
    "Salary":[30000,None,50000]
})
print(df.isna())
```

Output

```
    Name    Age  Salary
0  False  False   False
1   True  False    True
2  False   True   False
```

Explanation: isna() checks every cell and returns True where data is missing.

Easy interview explanation: Use isna() to detect missing values.

6. isna()

Code

```
result = df["Age"].isna()
print(result)
```

Output

```
0    False
1    False
2     True
```

```
Name: Age, dtype: bool
```

Explanation: isna() returns True for missing and False for non-missing values.

Easy interview explanation: isna() is a standard pandas missing-value check.

7. isnull()

Code

```
result = df["Age"].isnull()
print(result)
```

Output

```
0    False
1    False
2     True
Name: Age, dtype: bool
```

Explanation: isnull() works the same way as isna().

Easy interview explanation: isnull() is an alias for isna().

8. notna()

Code

```
result = df["Age"].notna()
print(result)
```

Output

```
0     True
1     True
2    False
Name: Age, dtype: bool
```

Explanation: notna() returns True when a value is present and False when it is missing.

Easy interview explanation: Use notna() to select rows with available values.

9. notnull()

Code

```
result = df["Age"].notnull()
print(result)
```

Output

```
0     True
1     True
2    False
Name: Age, dtype: bool
```

Explanation: notnull() works the same way as notna().

Easy interview explanation: notnull() is an alias for notna().

10. Count Missing Values

Code

```
print(df.isna().sum())
```

Output

```
Name      1
Age       1
Salary    1
dtype: int64
```

Explanation: sum() counts True as 1, so it counts missing values in each column.

Easy interview explanation: df.isna().sum() is a very useful data-quality check.

11. Total Missing Values

Code

```
total_missing = df.isna().sum().sum()
print(total_missing)
```

Output

```
3
```

Explanation: The first sum counts by column; the second sum adds all column counts.

Easy interview explanation: Use df.isna().sum().sum() for total missing cells.

12. Remove Missing Values — dropna()

Code

```
df = pd.DataFrame({"Name":["Amit","Riya","John"],"Age":[22,None,30]})
result = df.dropna()
print(result)
```

Output

```
   Name   Age
0  Amit  22.0
2  John  30.0
```

Explanation: dropna() removes rows containing missing values.

Easy interview explanation: Use dropna() when incomplete rows should be excluded.

13. dropna(how='all')

Code

```
df = pd.DataFrame({"Name":["Amit",None,"John"],"Age":[22,None,30]})
result = df.dropna(how="all")
print(result)
```

Output

```
   Name   Age
0  Amit  22.0
2  John  30.0
```

Explanation: how='all' removes a row only when every value in that row is missing.

Easy interview explanation: This keeps rows that contain at least one useful value.

14. Fill Missing Values — fillna()

Code

```
df = pd.DataFrame({"Age":[22,None,30]})
df["Age"] = df["Age"].fillna(25)
print(df)
```

Output

```
    Age
0  22.0
1  25.0
2  30.0
```

Explanation: fillna() replaces missing values with a value you provide.

Easy interview explanation: Use fillna() when you want to keep rows but replace missing values.

15. Fill with Mean

Code

```
df = pd.DataFrame({"Salary":[30000,None,50000]})
mean_salary = df["Salary"].mean()
df["Salary"] = df["Salary"].fillna(mean_salary)
print(df)
```

Output

```
    Salary
0  30000.0
1  40000.0
2  50000.0
```

Explanation: The mean is the average. Here, (30000 + 50000) / 2 = 40000.

Easy interview explanation: Mean filling can be useful for numeric data when outliers are not a major concern.

16. Fill with Median

Code

```
df = pd.DataFrame({"Salary":[30000,35000,None,100000]})
median_salary = df["Salary"].median()
df["Salary"] = df["Salary"].fillna(median_salary)
print(df)
```

Output

```
     Salary
0   30000.0
1   35000.0
2   35000.0
3  100000.0
```

Explanation: Median is the middle value after sorting and is less affected by extreme values.

Easy interview explanation: Median filling is often preferred when numeric data has outliers.

17. Fill with Mode

Code

```
df = pd.DataFrame({"City":["Pune","Mumbai",None,"Pune"]})
mode_city = df["City"].mode()[0]
df["City"] = df["City"].fillna(mode_city)
print(df)
```

Output

```
     City
0    Pune
1  Mumbai
2    Pune
3    Pune
```

Explanation: Mode is the most frequently occurring value.

Easy interview explanation: Mode filling is commonly used for categorical or text columns.

18. Forward Fill

Code

```
df = pd.DataFrame({"Sales":[100,None,None,400]})
df["Sales"] = df["Sales"].ffill()
print(df)
```

Output

```
   Sales
0  100.0
1  100.0
2  100.0
3  400.0
```

Explanation: Forward fill copies the previous available value forward.

Easy interview explanation: Use forward fill when the previous observation should continue.

19. Backward Fill

Code

```
df = pd.DataFrame({"Sales":[100,None,None,400]})
df["Sales"] = df["Sales"].bfill()
print(df)
```

Output

```
   Sales
0  100.0
1  400.0
2  400.0
3  400.0
```

Explanation: Backward fill copies the next available value backward.

Easy interview explanation: Use backward fill when the next observation is appropriate.

20. Interpolation

Code

```
df = pd.DataFrame({"Sales":[100,None,None,400]})
df["Sales"] = df["Sales"].interpolate()
print(df)
```

Output

```
   Sales
0  100.0
1  200.0
2  300.0
3  400.0
```

Explanation: Interpolation estimates missing values between known values.

Easy interview explanation: Interpolation is useful for ordered numeric or time-series data.

21. Complete Example

Code

```
df = pd.DataFrame({
    "Name":["Amit","Riya","John","Neha"],
    "Age":[22,None,30,25],
    "Salary":[30000,40000,None,35000]
})

print(df.isna().sum())

df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())

print(df)
```

Output

```
Name      0
Age       1
Salary    1
dtype: int64
   Name   Age   Salary
0  Amit  22.0  30000.0
1  Riya  25.0  40000.0
2  John  30.0  35000.0
3  Neha  25.0  35000.0
```

Explanation: First detect and count missing values, then choose an appropriate filling method.

Easy interview explanation: A practical workflow is detect → count → decide → drop or fill → verify.

LEVEL 9 QUICK REVISION

```
NaN / None / pd.NA -> Missing values
isna()             -> Detect missing
isnull()           -> Same as isna()
notna()            -> Detect available
notnull()          -> Same as notna()
isna().sum()       -> Count missing by column
dropna()           -> Remove missing data
fillna()           -> Replace missing data
mean()             -> Average
median()           -> Middle value
mode()             -> Most frequent value
ffill()            -> Forward fill
bfill()            -> Backward fill
interpolate()      -> Estimate between known values
```

Important: Do not blindly fill missing values. First understand why the data is missing, then choose the appropriate method.